

Lecture 17

Records, Fold, Recursion

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem
Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Elm Resources

- References
 - <https://guide.elm-lang.org/>
 - <https://elmprogramming.com/>
 - <https://elm-lang.org/examples>
- Online editors
 - <https://ellie-app.com/>
 - <https://elm-lang.org/try>
 - <https://elm-editor.com/>

Dictionaries

Dict: Stores key-value pairs (like a Python `dict`)

- Immutable! Inserting/updating/deleting returns a new *Dict*

Dictionaries: Functions

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
empty	<code>Dict k v</code>	Gives an empty dictionary	<code>empty</code>
singleton	<code>cmp → v → Dict cmp v</code>	Gives a dictionary with a single pair key + value	<code>singleton</code> <code>"a" 1</code>

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
fromList	List (cmp, v) → Dict cmp v	Makes dictionary from list of key-value pairs	fromList [("a", 1), ("b", 2)]
toList	Dict k v → List (k, v)	Turns dict into list of key-value pairs	toList (fromList [("a", 1), ("b", 2)])

Dictionaries: Functions

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
get	<code>cmp → Dict cmp</code> <code>v → Maybe v</code>	Gets value in dict associated with key (if exists)	<code>get</code> <code>"b" (fromList</code> <code>[("a", 1),</code> <code>("b", 2)])</code>
insert	<code>cmp → v → Dict</code> <code>cmp v → Dict</code> <code>cmp v</code>	Adds/ Overwrites the	<code>insert "b" 2</code> <code>(fromList</code> <code>[("a", 1)])</code>

Function Signature

Description

Usage

value of key in
dict with value

remove `cmp → Dict cmp`
 `v → Dict cmp v`

Removes key
and its value in
dict

```
remove  
"b" (fromList  
[("a", 1),  
 ("b", 2)])
```


Dictionaries: Functions

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
isEmpty	<code>Dict k v → Bool</code>	Determines emptiness of dict	<code>isEmpty empty</code>
member	<code>cmp → Dict cmp v → Bool</code>	Determines existence of key in dict	<code>member "a" (fromList [("a", 1)])</code>

Dictionaries: Functions

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
size	<code>Dict k v → Int</code>	Gives number of key-value pairs in dict	<code>size (fromList [("a", 1), ("b", 2)])</code>
keys	<code>Dict k v → List k</code>	Gives list of keys of dict	<code>keys (fromList [("a", 1), ("b", 2)])</code>
values	<code>Dict k v → List v</code>	Gives list of values of dict	<code>values (fromList</code>

Function Signature

Description

Usage

```
[("a", 1),  
 ("b", 2)]
```

Records

Record: Collection of values with *fixed key names*

- **Dict** key names are dynamic
- Type definition:

```
type alias Class =  
  { section : String,  
    , students : Int,  
    , handler : String  
  }
```

Records

Record: Collection of values with *fixed key names*

- **Dict** key names are dynamic
- Creation:

```
x =  
    {section = "CS 12 FRU1"  
      , students = 19  
      , handler = "Martinez"  
    }
```

Records

- Updating subset of fields

```
x =  
  {section = "CS 12 FRU1"  
    , students = 19  
    , handler = "Martinez"  
  }
```

```
y = {x | students = x.students + 1}
```

Tabular data processing

- Common Tabular Data Formats
 - CSV (comma-separated-values)
 - TSV (tab-separated values)
 - Spreadsheets (.xlsx, .xls, .ods, etc)
 - Databases (CS 165)

Common tabular data representations

- Container of containers
- Dictionaries
- Records

Simple tabular data example

- Comma-separated values as string:

```
-- Note the leading \n at the start of the string
csv = """
Section,Students,Handler
CS 12 LAB 1,19,Coronel
CS 12 LAB 2,15,Beltran
CS 12 LAB 3,17,Addawe
CS 12 LAB 4,15,Labao
CS 12 LAB 5,19,Coronel
CS 12 LAB 6,19,Beltran
CS 12 LAB 7,19,Addawe
```

CS 12 LAB 8,19,Labao"""

- First row may contain column names

Records representation

- Count the number of students:

```
-- Note the leading \n at the start of the string
csv = """
Section,Students,Handler
CS 12 LAB 1,19,Coronel
CS 12 LAB 2,15,Beltran
CS 12 LAB 3,17,Addawe
CS 12 LAB 4,15,Labao
CS 12 LAB 5,19,Coronel
CS 12 LAB 6,19,Beltran
CS 12 LAB 7,19,Addawe
```

CS 12 LAB 8,19,Labao"""

- Hint: check the Elm cheatsheet for usable functions!
- Hint: array access

For loop examples

```
nums: list[int] = [10, 20, 30, ...]  
summy: int = 0  
N: int = len(nums)  
  
for i in range(N):  
    summy = summy + nums[i]  
  
# do something with summy
```

For loop examples

```
# Above can be rewritten as follows
nums: list[int] = [10, 20, 30, ...]
summy: int = 0
N: int = len(nums)

for i in range(N):
    summy = add(summy, nums[i])

# do something with summy
```

For loop examples

```
nums: list[int] = [10, 20, 30, ...]
product: int = 1
N: int = len(nums)

for i in range(N):
    product = product * nums[i]

# do something with product
```

For loop examples

```
# Above can be rewritten as follows
nums: list[int] = [10, 20, 30, ...]
product: int = 1
N: int = len(nums)

for i in range(N):
    product = mul(product, nums[i])

# do something with product
```


For loop examples

```
# Assumes nonnegative Ints
nums: list[int] = [10, 20, 30, ...]
max_so_far: int = 0
N: int = len(nums)

for i in range(N):
    max_so_far = max(max_so_far, nums[i])

# do something with max_so_far
```

For loop examples

```
# Assumes nonnegative Ints
nums: list[int] = [10, 20, 30, ...]
min_so_far: int = MAXX
N: int = len(nums)

for i in range(N):
    min_so_far = min(min_so_far, nums[i])

# do something with min_so_far
```

For loop examples

- Which parts of their structure are common to all of them?

For loop examples

<i>Operation</i>	<i>List of elements</i>	<i>Result variable</i>	<i>Starting value of result</i>	<i>Function used with each element</i>
Addition	nums	summy	0	add
Multiplication	nums	product	1	mul
Maximum	nums	max_so_far	0	max
Minimum	nums	min_so_far	MAXX	min

For loop examples

- Emerging pattern (pseudocode)

```
elems = [...]  
result = STARTING_VALUE  
  
for elem in elems:  
    result = FUNCTION(result, elem)
```

For loop examples

- Is there a common name for this pattern?

Fold

Fold: Abstraction of *for-each aggregation* pattern

- Also called *reduce*, *accumulate*, and *aggregate*

Fold

- *fold* as pseudocode:

```
def fold(REDUCER, STARTING_VALUE, elems):  
    result = STARTING_VALUE  
  
    for elem in elems:  
        result = REDUCER(elem, result)  
  
    return result
```


Earlier examples as folds

- Assume existence of reducers *add*, *mul*, *max*, and *min*

```
def fold(REDUCER, STARTING_VALUE, elems):  
    result = STARTING_VALUE  
  
    for elem in elems:  
        result = REDUCER(elem, result)  
  
    return result
```

Earlier examples as folds

- Assume existence of reducers *add*, *mul*, *max*, and *min*

```
def sum(nums):  
    return fold(add, 0, nums)
```

```
def product(nums):  
    return fold(mul, 1, nums)
```

```
def maximum(nums):  
    return fold(max, 0, nums)
```

```
def minimum(nums):
```

```
return fold(min, MAXX, nums)
```

Left and right folds

- Left fold

```
def fold(REDUCER, STARTING_VALUE, elems):  
    result = STARTING_VALUE  
  
    # leftmost first  
    for elem in elems:  
        result = REDUCER(elem, result)  
  
    return result
```

Left and right folds

```
def sum(nums):  
    return foldl(add, 0, nums)
```

```
# Leftmost first  
sum([1, 2, 3])
```

```
# 3rd      2nd      1st  
add(3, add(2, add(1, 0)))  
add(3, add(2, 1))  
add(3, 3)  
6
```

Left and right folds

- Right fold

```
def fold(REDUCER, STARTING_VALUE, elems):  
    result = STARTING_VALUE  
  
    # rightmost first; reverse order  
    for elem in reversed(elems):  
        result = REDUCER(elem, result)  
  
    return result
```

Left and right folds

- Right fold

```
def sum(nums):  
    return foldr(add, 0, nums)
```

```
# Rightmost first  
sum([1, 2, 3])
```

```
# 3rd      2nd      1st  
add(1, add(2, add(3, 0)))  
add(1, add(2, 3))  
add(1, 5)
```


Literal sense of folding

<https://cs12222.upd-dcs.work/lecture08/img/literalfold.gif>

Left and right folds

- Order matters for noncommutative reducers

```
def concat(strs):  
    return foldl(append, "", strs)  
  
# Leftmost first  
concat(["a", "b", "c"])  
  
# 3rd          2nd          1st  
append("c", append("b", append("a", "")))  
append("c", append("b", "a"))  
append("c", "ba")
```

"cba"

Left and right folds

- Order matters for noncommutative reducers

```
def concat(strs):  
    return foldr(append, "", strs)  
  
# Rightmost first  
concat(["a", "b", "c"])  
  
# 3rd          2nd          1st  
append("a", append("b", append("c", "")))  
append("a", append("b", "c"))  
append("a", "bc")
```

"abc"

Left and right folds

- Starting value is second argument

```
def concat(strs):  
    return foldl(append, "Z", strs)  
  
# Leftmost first  
concat(["a", "b", "c"])  
  
# 3rd          2nd          1st  
append("c", append("b", append("a", "")))  
append("c", append("b", "a"))  
append("c", "ba")
```

"cbaZ"

Left and right folds

- Starting value is second argument

```
def concat(strs):  
    return foldr(append, "Z", strs)  
  
# Rightmost first  
concat(["a", "b", "c"])  
  
# 3rd          2nd          1st  
append("a", append("b", append("c", "")))  
append("a", append("b", "c"))  
append("a", "bc")
```


"abcZ"

List Fold in Elm

- Left fold and right fold in `List` module:

```
-- Same signature
```

```
foldl : (a -> b -> b) -> b -> List a -> b
```

```
foldr : (a -> b -> b) -> b -> List a -> b
```

```
--          REDUCER                -> STARTING_VALUE -> elems -> result
```

```
foldl : (a -> b -> b) -> b -> List a -> b
```

```
foldr : (a -> b -> b) -> b -> List a -> b
```

List Fold in Elm

- `Array`, `Dict`, `Set`, and `String` have folds as well

Earlier examples as Elm folds

```
-- May replace `foldl` with `foldr` if commutative
```

```
sum : List Int -> Int
```

```
sum nums = List.foldl add 0 nums
```

```
product : List Int -> Int
```

```
product nums = List.foldl mul 1 nums
```

```
nonNegativeMaximum : List Int -> Int
```

```
nonNegativeMaximum nums = List.foldl max 0 nums
```

```
nonNegativeMinimum : List Int -> Int
```

```
nonNegativeMinimum nums = List.foldl min MAXX nums
```

```
concat : List String -> String
```

```
concat strs = List.foldl append "" strs
```

Reduction

Reduction: Transformation from `List a` to `a`

- List is “reduced” to a single value
- Output has same type as container elements

```
sum           : List Int -> Int
product       : List Int -> Int
nonNegativeMaximum : List Int -> Int
nonNegativeMinimum : List Int -> Int
concat        : List String -> String
```

Reduction

- Reducer is binary function with same types:

```
add    : Int -> Int -> Int
mul    : Int -> Int -> Int
min    : Int -> Int -> Int
max    : Int -> Int -> Int
append : String -> String -> String
```

Reduction

Identity element: IDENTITY for which reducer returns other argument

- Set as default value of reduction; covers single-element case
- Left identity: `reducer IDENTITY x == x`
- Right identity: `reducer x IDENTITY == x`

Reduction

```
add : Int -> Int -> Int
```

```
add a b = a + b
```

```
-- IDENTITY is 0: add a 0 == a + 0 == a
```

```
mul : Int -> Int -> Int
```

```
mul a b = a * b
```

```
-- IDENTITY is 1: mul a 1 == a * 1 == a
```

```
append : String -> String -> String
```

```
append a b = a ++ b
```

```
-- append x IDENTITY == x ++ IDENTITY == x
```



```
-- Self-check: What is `IDENTITY` for `append`?
```

Accumulation

Accumulation: Transformation from **List** a to b

- Output is any type
- More general than reduction
- No change in fold pseudocode

Intermediate state

Task: Create a function `extend : String -> String` that repeats the *n*th character *n* times for all characters of the input string

- Example: `extend "abc" == "abbccc"`

Intermediate state

- Loop-based solution:

```
def extend(s):  
    answer = ""  
    k = 1          # `k` is intermediate state; tracks  
                   # current position in input string  
  
    # For each character in `s`  
    for ch in s:  
        answer = answer + (ch * k)  # Repeat `k` times  
        k = k + 1
```

```
return answer
```

Intermediate state

- How do we express intermediate state in folds?

Intermediate state

```
def foldl( Reducer , Initial accumulated answer , Element , Element , ... ) :
```

```
    Accumulator = ( Initial accumulated answer , Initial accumulated intermediate state )
```

```
    for Element in Element , Element , ... :
```

Accumulator

```
        Accumulator = Reducer ( Element , ( Accumulated answer , Accumulated intermediate state ) )
```

```
    return Accumulated answer
```

Intermediate state

- Default value is now a *pair* (tuple with two elements)
- Return value is first element of pair

Intermediate state

```
# `extend` rewritten in `foldl` template
def extend(elems):
    acc = ("", 1) # (answer, k)

    for elem in elems:
        # acc = reducer(elem, acc)
        (answer, k) = acc
        acc = (answer + (elem * k), k + 1)

    return answer
```

Intermediate state

```
def reducer(ch, acc):                # Same as what
    (answer, k) = acc                # loop body does
    return (answer + (ch * k), k + 1)
```

```
                                # (answer, k)
reducer('a', ("", 1)) == ("a", 2)
reducer('b', ("a", 2)) == ("abb", 3)
reducer('c', ("abb", 3)) == ("abbccc", 4)
```

```
def extend(s):
    (answer, k) = foldl(reducer, ("", 1), s)
```

```
return answer
```

Aside: Elm String folding

```
f : Char -> String -> String
f elem acc =
    (elem |> Char.toUpperCase |> String.fromCharCode) ++ acc
```

- Elm distinguishes between single Characters and Strings

extend in Elm

```
extend : String -> String
extend str =
    let
        reducer : Char -> (String, Int) -> (String, Int)
        reducer ch acc =
            let
                (answer, k) = acc
                nextAnswer = answer ++ String.repeat k
                    (String.fromCharCode ch)
                nextK = k + 1
            in
```

```
        (nextAnswer, nextK)  
    (finalAns, _) = String.foldl reducer ("", 1) str  
in  
finalAns
```

Practice

- Try to implement all `String` module functions using only `foldl` and `foldr` of `String` and `List` (as well as `++`)

Practice

- Suggestions:
 - `String.length`
 - `String.left` and `String.right`
 - `String.dropLeft` and `String.dropRight`
 - `String.fromList`
 - `String.reverse`
 - `String.startsWith` and `String.endsWith`